POC-03 - Real-Time Data Platform

Event Hubs, Databricks, Fabric and Synapse

Beginner implementation and troubleshooting guide

Manual setup, Python commands, streaming validation, SQL/KQL, issues and fixes

How to Use This Guide

This guide reconstructs the completed POC as a repeatable beginner runbook. Follow the sections in order the first time. On later runs, use the troubleshooting, command sequence, and validation checklist as a quick reference.

Section	What it covers
1. Foundation	Architecture, prerequisites, project structure, event schema
2. Azure setup	Resource group, Event Hubs, ADLS Gen2, Databricks
3. Local ingestion	Virtual environment, .env, send_events.py, receive_events.py
4. Databricks streaming	Bronze/Silver, checkpoints, watermark, dedup, Serverless fixes
5. Gold and SQL	Gold KPIs, Unity Catalog/SQL views, validation queries
6. Fabric and Synapse	Eventstream, Eventhouse/KQL, OneLake, serverless SQL reference path
7. Operations	Monitoring, Purview notes, cleanup, final validation, interview Q&A;

How to Use This Guide - Continued

Evidence convention

Screenshots show the actual POC run where supplied. Where the repository contains a reference branch without a screenshot, the guide labels it as a reference implementation step rather than claiming screenshot evidence.

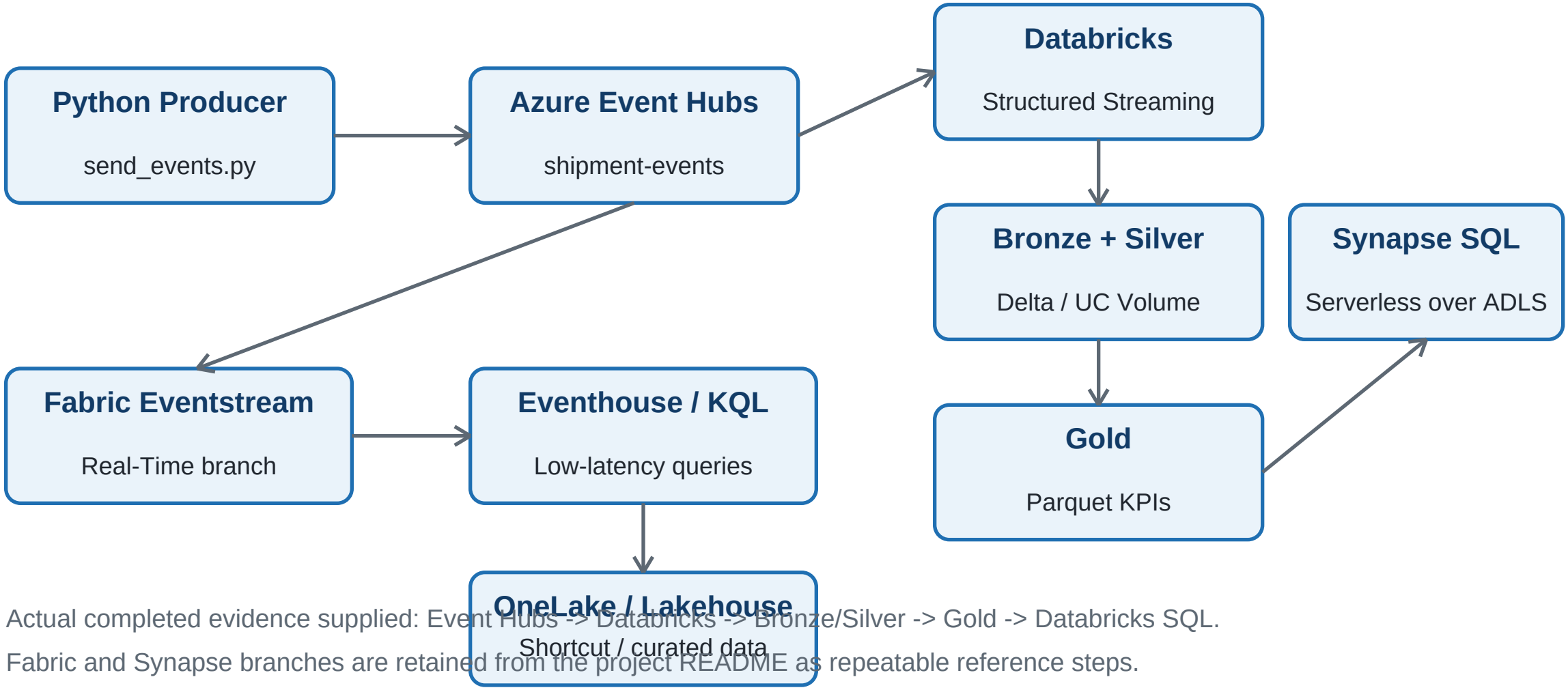
POC Success State - What Was Actually Proven

- Azure resource group, Event Hubs namespace, storage account, and Azure Databricks workspace were created.
- The Python producer successfully sent shipment events to Azure Event Hubs, and the local receiver successfully read them.
- The Databricks stream ultimately ran successfully on Serverless compute using a Unity Catalog Volume-compatible approach.
- A successful evidence run wrote 50 raw Bronze events.
- Silver validation produced 48 clean events with duplicate count 0.
- Gold KPI aggregation was exported successfully from the 48 clean Silver events.
- Databricks SQL views were created over the Silver Delta path and Gold Parquet path.

Important architecture difference

The original repository design writes curated data to ADLS with abfss:// paths. The successful Serverless run in the screenshots was adapted to /Volumes/dbw_logistics_poc/default/realtime. Synapse serverless SQL can query the repository ADLS path, but it cannot directly use the Databricks managed Volume path. To repeat the Synapse branch, publish Gold to ADLS first.

Architecture - What You Are Building



Actual completed evidence supplied: Event Hubs -> Databricks -> Bronze/Silver -> Gold -> Databricks SQL.

Fabric and Synapse branches are retained from the project README as repeatable reference steps.

Learning Objectives and Cost Guardrails

You should be able to explain

- What Azure Event Hubs does and why partition keys matter.
- How a Python producer sends JSON events and how a consumer verifies arrival.
- How Databricks Structured Streaming reads the Event Hubs Kafka-compatible endpoint.
- Checkpoint vs watermark, duplicate handling, late-event handling, and restart behavior.
- Bronze, Silver, and Gold layers and why curated Gold data is useful for BI/SQL.
- Eventstream/Eventhouse/KQL vs Lakehouse/Delta use cases.
- OneLake shortcut vs copy, Synapse serverless SQL, semantic measures, monitoring, and Purview governance.

Cost guardrails

- Send only a few hundred events for learning.
- Use minimal Databricks compute and stop or auto-terminate it when finished.
- Use Fabric Trial/capacity only if available for your account.
- Use Synapse serverless SQL only; do not create a dedicated SQL pool for this POC.
- Keep Purview/Log Analytics optional if account availability or cost is unclear.

Project Structure

Repository layout

```
POC_03_REALTIME_FABRIC_SYNAPSE/
|--  .env.example
|--  README.md
|--  requirements.txt
|--  producer/
|   |--  send_events.py
|   \--  receive_events.py
|--  databricks/
|   |--  stream_to_delta.py
|   |--  inspect_and_validate.py
|   \--  export_gold_parquet.py
|--  fabric/setup_notes.md
|--  kql/create_shipment_table.kql
|--  kql/shipment_queries.kql
|--  synapse/serverless_queries.sql
|--  semantic_model/measures.dax
|--  governance/purview_notes.md
|--  monitoring/monitoring_checklist.md
|--  docs/azure_portal_setup.md
|--  docs/troubleshooting.md
```

Project Structure - Continued

```
|--  docs/cleanup.md
\--  evidence/README.md
```

Prerequisites - Before You Create Anything

- Azure subscription with permission to create Resource Groups, Storage Accounts, Event Hubs, Databricks, and Synapse workspaces.
- Python 3.9 or newer and pip on the local machine.
- A browser for Azure portal and Databricks workspace access.
- Microsoft Fabric workspace with supported capacity or Trial only if you execute the Fabric branch.
- Recommended local tools: VS Code and Git. Azure CLI is optional for the manual portal-first path.

Beginner rule

Do not start Databricks or Fabric until local Event Hubs send/receive verification works. This reduces the number of systems you must debug at the same time.

Event Schema

JSON event example

```
{
  "event_id": "evt-001",
  "order_id": 1001,
  "event_type": "SHIPPED",
  "event_ts": "2026-08-28T10:00:00Z",
  "region": "IN-SOUTH",
  "revenue": 1299.50,
  "fulfillment_minutes": 120,
  "producer_seq": 1
}
```

- event_id is the deduplication key used in Silver.
- event_ts is the event-time field used for watermark behavior.
- region is also used by the producer as the partition key.
- revenue and fulfillment_minutes support Gold KPIs and semantic-model measures.

Recommended Execution Order

Order	Action	Pass condition
1	Create resource group	Resource group opens in Azure portal
2	Create Event Hubs namespace + shipment-events	Event Hub entity exists
3	Create ADLS Gen2 storage + realtime container	Container/filesystem exists
4	Configure local .env	Python can read the connection string
5	Run producer and receiver	Events send and receive successfully
6	Create Databricks and import scripts	Workspace/notebooks open
7	Run Bronze/Silver stream	Rows written and Delta files created
8	Validate Silver	No duplicate event_id values
9	Build Gold	Gold KPI summary exported
10	Create SQL views / queries	Curated queries execute
11	Fabric/Synapse branch if required	KQL/OPENROWSET validation succeeds

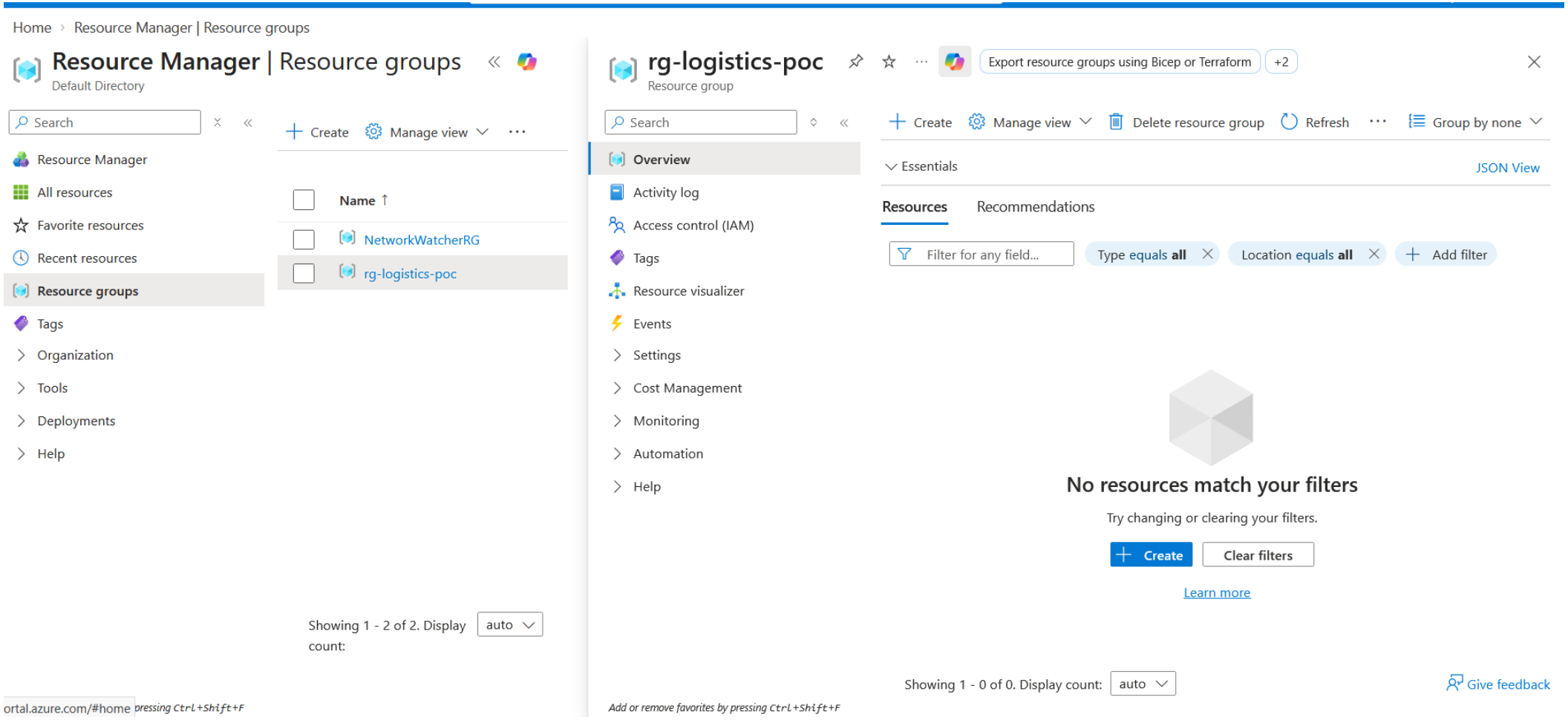
Step 1 - Create the Azure Resource Group

- Open Azure portal and search for Resource groups.
- Select Create.
- Choose your Azure subscription.
- Use a dedicated POC resource group. The completed run used rg-logistics-poc.
- Choose the Azure region you intend to use for the POC resources where practical.
- Select Review + create, then Create.

Verify

Open the resource group after deployment. It should exist before you create the remaining services.

Evidence - Resource Group Created



The completed run shows the dedicated resource group rg-logistics-poc in Azure Resource Manager.

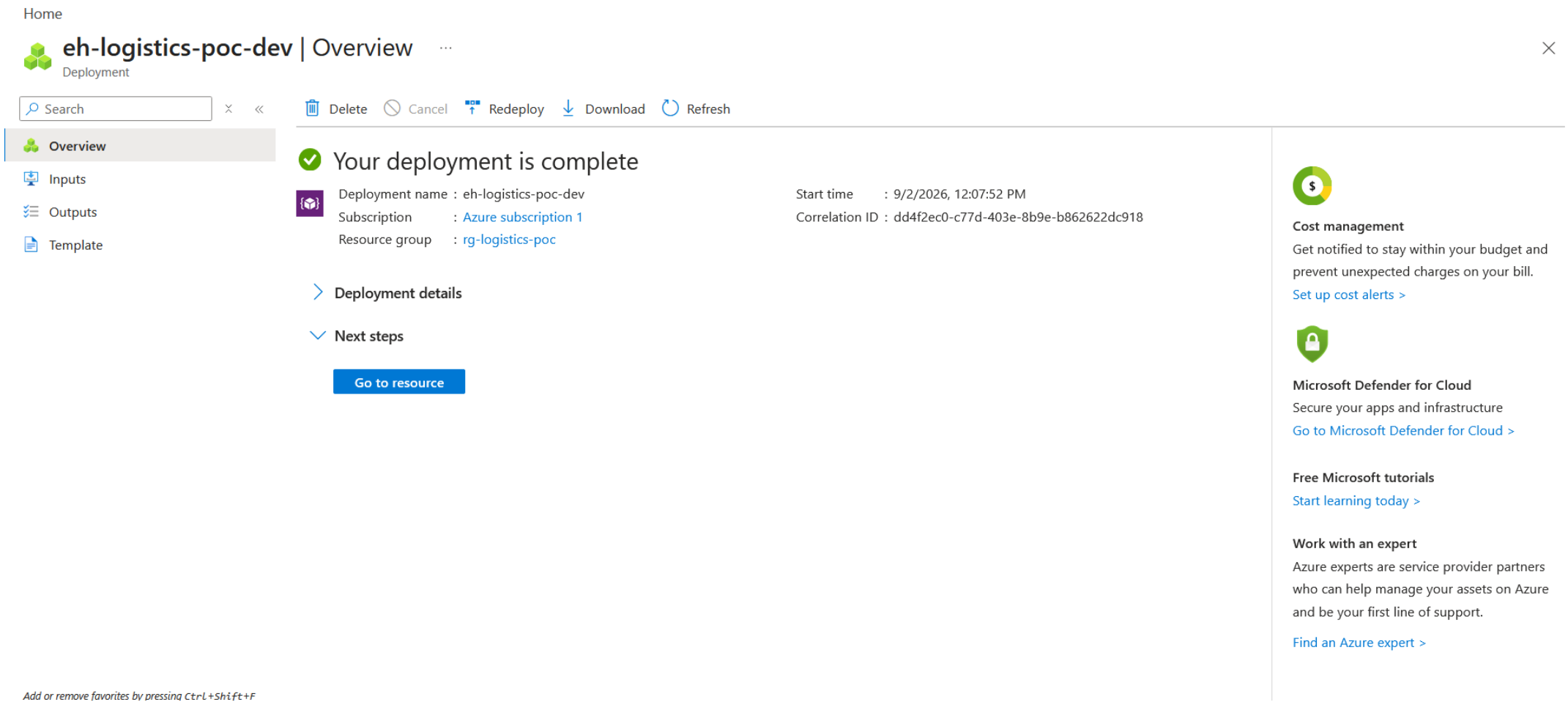
Step 2 - Create the Event Hubs Namespace

- Search Azure portal for Event Hubs and create a namespace.
- Select the existing POC resource group.
- Use a globally unique namespace name. The completed run used an eh-logistics-poc-dev style name.
- Keep throughput/capacity minimal for the POC.
- The raw setup notes used Standard for the Kafka/multiple-consumer-group learning path.
- Create the namespace and wait for deployment to complete.

Then create the Event Hub entity

- Open the namespace -> Event Hubs -> + Event Hub.
- Name the entity shipment-events.
- Use a small/default partition count and minimal/default retention for the POC.

Evidence - Event Hubs Namespace



The Event Hubs namespace deployment completed successfully.

Event Hubs Authentication and Consumer Group

Connection string for the local POC

- Namespace -> Shared access policies.
- Use a POC policy with the rights required by your producer/consumer. The raw run used the namespace shared access policy path.
- Copy Connection string-primary key, not only the Primary key value.
- Store the connection string in .env. Never commit the real secret.

Event Hub values

Environment values

```
EVENT_HUB_NAME=shipment-events
EVENT_HUB_CONSUMER_GROUP=$Default
EVENT_HUB_NAMESPACE=<your-namespace>
EVENT_HUB_KAFKA_BOOTSTRAP=<your-namespace>.servicebus.windows.net:9093
```

Production security

For a production design, prefer Microsoft Entra ID / managed identity where supported rather than long-lived shared keys.

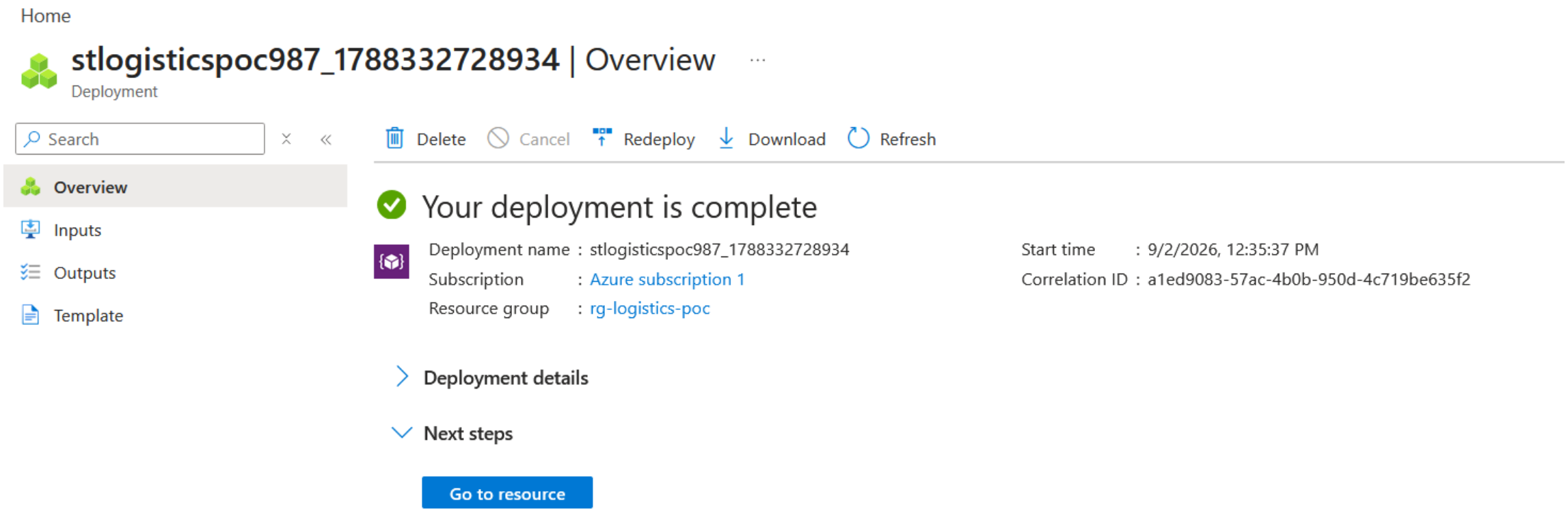
Step 3 - Create ADLS Gen2 Storage

- Azure portal -> Storage accounts -> Create.
- Use the same POC resource group.
- Choose a globally unique lowercase storage account name. The completed run used stlogisticspoc987.
- Performance: Standard.
- For the learning POC, the raw steps selected locally redundant storage (LRS).
- In Advanced settings, enable Hierarchical namespace. This turns on ADLS Gen2 semantics.
- Create the storage account.

Create the filesystem/container

- Storage account -> Data storage -> Containers -> + Container.
- Name it realtime.
- Keep anonymous access disabled/private.

Evidence - Storage Account Created



The storage account deployment completed successfully.

ADLS Paths and .env Storage Values

Repository design - ADLS paths

```
ADLS_ACCOUNT_NAME=stlogisticspoc987
ADLS_CONTAINER_NAME=realtime

DELTA_BRONZE_PATH=abfss://realtime@stlogisticspoc987.dfs.core.windows.net/
  delta/bronze/shipment_events
DELTA_SILVER_PATH=abfss://realtime@stlogisticspoc987.dfs.core.windows.net/
  delta/silver/shipment_events
GOLD_PARQUET_PATH=abfss://realtime@stlogisticspoc987.dfs.core.windows.net/
  gold/shipment_summary
```

Do not create every subfolder manually

Spark creates Delta/Parquet subdirectories when the write succeeds and permissions are correct.

Step 4 - Create the Local Python Environment

Windows Command Prompt / PowerShell

```
python -m venv .venv
.venv\Scripts\activate
python -m pip install --upgrade pip
pip install -r requirements.txt
copy .env.example .env
```

Linux / macOS / WSL activation

Linux / macOS / WSL

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
```

Required package

The project requirements install azure-eventhub and python-dotenv for the local producer/receiver path.

Complete .env Template - Safe Placeholder Version

.env - never place real secrets in shared documentation

```
EVENT_HUB_CONNECTION_STRING=Endpoint=sb://<namespace>.servicebus.windows.net/;
  SharedAccessKeyName=<policy>;SharedAccessKey=<key>
EVENT_HUB_NAME=shipment-events
EVENT_HUB_CONSUMER_GROUP=$Default

EVENT_COUNT=200
EVENT_DELAY_SECONDS=0.15
DUPLICATE_EVERY=25
LATE_EVENT_EVERY=40

ADLS_ACCOUNT_NAME=<storage-account-name>
ADLS_CONTAINER_NAME=realtime
DELTA_BRONZE_PATH=abfss://realtime@<storage-account-name>.dfs.core.windows.net/
  delta/bronze/shipment_events
DELTA_SILVER_PATH=abfss://realtime@<storage-account-name>.dfs.core.windows.net/
  delta/silver/shipment_events
GOLD_PARQUET_PATH=abfss://realtime@<storage-account-name>.dfs.core.windows.net/
  gold/shipment_summary

EVENT_HUB_NAMESPACE=<namespace>
```

Complete .env Template - Safe Placeholder Version - Continued

```
EVENT_HUB_KAFKA_BOOTSTRAP=<namespace>.servicebus.windows.net:9093
```

First Local Verification Commands

1. Confirm Python can read .env

PowerShell / CMD

```
python -c "import os; from dotenv import load_dotenv; load_dotenv();
print('Length of connection string:',
len(os.getenv('EVENT_HUB_CONNECTION_STRING') or ''))"
```

2. Send a small batch first

Command

```
python producer\send_events.py --count 20 --delay 0.2
```

3. Receive a few events

Command

```
python producer\receive_events.py --max-events 10 --starting-position -1
```

First Local Verification Commands - Continued

Pass condition

Do not move to Databricks until both sender and receiver work locally.

Issue 1 - getaddrinfo Failed / Placeholder Namespace

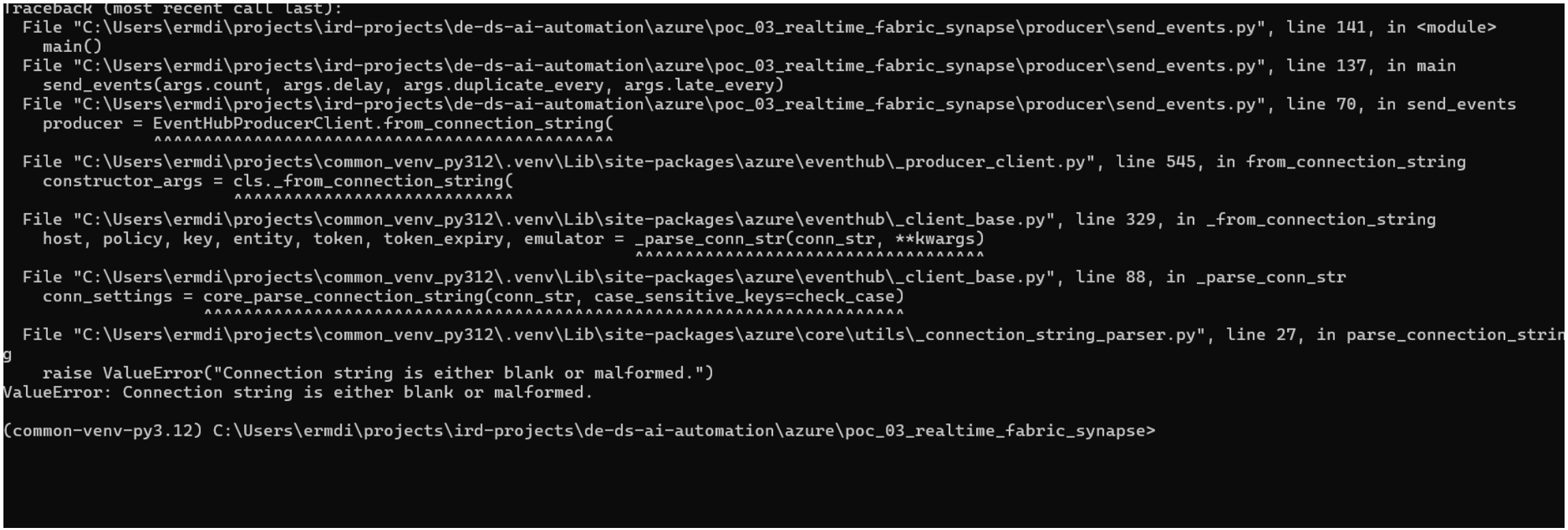
Symptom

Python cannot resolve the host because .env still contains a placeholder such as .servicebus.windows.net.

Fix

- Open your Event Hubs namespace in Azure portal and copy the exact namespace name.
- Set EVENT_HUB_NAMESPACE to that exact value.
- Set EVENT_HUB_KAFKA_BOOTSTRAP to .servicebus.windows.net:9093.
- Replace the placeholder connection string with the real Connection string-primary key.
- Save .env and rerun the producer.

Issue 2 - Connection String Is Blank or Malformed



```
Traceback (most recent call last):
  File "C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\poc_03_realtime_fabric_synapse\producer\send_events.py", line 141, in <module>
    main()
  File "C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\poc_03_realtime_fabric_synapse\producer\send_events.py", line 137, in main
    send_events(args.count, args.delay, args.duplicate_every, args.late_every)
  File "C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\poc_03_realtime_fabric_synapse\producer\send_events.py", line 70, in send_events
    producer = EventHubProducerClient.from_connection_string(
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\ermdi\projects\common_venv_py312\.venv\Lib\site-packages\azure\eventhub\_producer_client.py", line 545, in from_connection_string
    constructor_args = cls._from_connection_string(
                       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\ermdi\projects\common_venv_py312\.venv\Lib\site-packages\azure\eventhub\_client_base.py", line 329, in _from_connection_string
    host, policy, key, entity, token, token_expiry, emulator = _parse_conn_str(conn_str, **kwargs)
                                                               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\ermdi\projects\common_venv_py312\.venv\Lib\site-packages\azure\eventhub\_client_base.py", line 88, in _parse_conn_str
    conn_settings = core_parse_connection_string(conn_str, case_sensitive_keys=check_case)
                    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\ermdi\projects\common_venv_py312\.venv\Lib\site-packages\azure\core\utils\_connection_string_parser.py", line 27, in parse_connection_strin
g
    raise ValueError("Connection string is either blank or malformed.")
ValueError: Connection string is either blank or malformed.

(common-venv-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\poc_03_realtime_fabric_synapse>
```

Terminal evidence from the failed local producer run. Personal local-path details are cropped/anonymized.

Issue 2 - Connection String Is Blank or Malformed - Continued

Primary cause from the run

Python either could not load EVENT_HUB_CONNECTION_STRING from .env or the value was not a valid Event Hubs connection string.

Fix - Validate the Event Hubs Connection String

Check 1 - Copy the right field

- Wrong: only the random-looking Primary key.
- Correct: Connection string-primary key beginning with Endpoint=sb://.

Check 2 - Verify the Windows filename

PowerShell

```
Get-ChildItem -Hidden .env*
# If the file is actually .env.txt:
Rename-Item .env.txt .env
```

Check 3 - Keep .env formatting simple

Correct pattern

```
EVENT_HUB_CONNECTION_STRING=Endpoint=sb://...
# Avoid extra spaces around the variable name and avoid accidental comments.
```

Check 4 - Test the loaded length

Fix - Validate the Event Hubs Connection String - Continued

Diagnostic command

```
python -c "import os; from dotenv import load_dotenv; load_dotenv();
print(len(os.getenv('EVENT_HUB_CONNECTION_STRING') or ''))"
```

Success - Producer Sent the Event Batch

```
[168/200] NORMAL    id=evt-9a739fb158ce type=SHIPPED    region=IN-WEST ts=2026-09-02T07:19:04Z
[169/200] NORMAL    id=evt-4389556967d2 type=PACKED     region=IN-EAST ts=2026-09-02T07:19:05Z
[170/200] NORMAL    id=evt-a9576821da96 type=DELAYED    region=IN-WEST ts=2026-09-02T07:19:05Z
[171/200] NORMAL    id=evt-037c0b970c9b type=SHIPPED    region=IN-NORTH ts=2026-09-02T07:19:06Z
[172/200] NORMAL    id=evt-7eb72f74fee9 type=PACKED     region=IN-EAST ts=2026-09-02T07:19:06Z
[173/200] NORMAL    id=evt-a48492969b00 type=CREATED    region=IN-WEST ts=2026-09-02T07:19:06Z
[174/200] NORMAL    id=evt-afc9f3a5cbc7 type=DELIVERED  region=IN-EAST ts=2026-09-02T07:19:07Z
[175/200] DUPLICATE id=evt-afc9f3a5cbc7 type=DELIVERED  region=IN-EAST ts=2026-09-02T07:19:07Z
[176/200] NORMAL    id=evt-b1494312d311 type=SHIPPED    region=IN-SOUTH ts=2026-09-02T07:19:07Z
[177/200] NORMAL    id=evt-0547d6932fb3 type=DELAYED    region=IN-NORTH ts=2026-09-02T07:19:08Z
[178/200] NORMAL    id=evt-83806a05ec90 type=CREATED    region=IN-SOUTH ts=2026-09-02T07:19:08Z
[179/200] NORMAL    id=evt-577d1f2ad115 type=DELIVERED  region=IN-EAST ts=2026-09-02T07:19:09Z
[180/200] NORMAL    id=evt-a17fa52f752e type=PACKED     region=IN-SOUTH ts=2026-09-02T07:19:09Z
[181/200] NORMAL    id=evt-369c9c532d4f type=IN_TRANSIT region=IN-WEST ts=2026-09-02T07:19:09Z
[182/200] NORMAL    id=evt-08bcf099bbbf type=IN_TRANSIT region=IN-NORTH ts=2026-09-02T07:19:10Z
[183/200] NORMAL    id=evt-6f04c50ba215 type=PACKED     region=IN-SOUTH ts=2026-09-02T07:19:10Z
[184/200] NORMAL    id=evt-1d20a96e32af type=PACKED     region=IN-EAST ts=2026-09-02T07:19:11Z
[185/200] NORMAL    id=evt-95975e9ad9bb type=DELAYED    region=IN-WEST ts=2026-09-02T07:19:11Z
[186/200] NORMAL    id=evt-a3309418b466 type=PACKED     region=IN-NORTH ts=2026-09-02T07:19:11Z
[187/200] NORMAL    id=evt-633a01df7738 type=DELIVERED  region=IN-NORTH ts=2026-09-02T07:19:12Z
[188/200] NORMAL    id=evt-ee02e20ac5b0 type=PACKED     region=IN-EAST ts=2026-09-02T07:19:12Z
[189/200] NORMAL    id=evt-a5161ac6a187 type=SHIPPED    region=IN-SOUTH ts=2026-09-02T07:19:13Z
[190/200] NORMAL    id=evt-184820e1a25f type=DELIVERED  region=IN-WEST ts=2026-09-02T07:19:13Z
[191/200] NORMAL    id=evt-8073c63e6ecd type=SHIPPED    region=IN-NORTH ts=2026-09-02T07:19:13Z
[192/200] NORMAL    id=evt-d267c573fea8 type=SHIPPED    region=IN-WEST ts=2026-09-02T07:19:14Z
[193/200] NORMAL    id=evt-93f37ab95dad type=PACKED     region=IN-SOUTH ts=2026-09-02T07:19:14Z
[194/200] NORMAL    id=evt-000b2a08bd99 type=CREATED    region=IN-WEST ts=2026-09-02T07:19:15Z
[195/200] NORMAL    id=evt-0264ddf8604e type=DELIVERED  region=IN-SOUTH ts=2026-09-02T07:19:15Z
[196/200] NORMAL    id=evt-c264682cdff3 type=SHIPPED    region=IN-EAST ts=2026-09-02T07:19:15Z
[197/200] NORMAL    id=evt-4ed1aa278289 type=DELIVERED  region=IN-WEST ts=2026-09-02T07:19:16Z
[198/200] NORMAL    id=evt-5ecaabd51a4e type=SHIPPED    region=IN-SOUTH ts=2026-09-02T07:19:16Z
[199/200] NORMAL    id=evt-94918d3c3d33 type=PACKED     region=IN-EAST ts=2026-09-02T07:19:17Z
[200/200] DUPLICATE id=evt-94918d3c3d33 type=PACKED     region=IN-EAST ts=2026-09-02T07:19:17Z

Summary
  sent attempts : 200
  duplicates    : 8
  late events   : 4
Check Event Hubs > Monitoring > Metrics > Incoming Messages.
```

The producer completed successfully and printed NORMAL, DUPLICATE, and LATE event markers.

Success - Producer Sent the Event Batch - Continued

Observed behavior

The raw run confirmed a successful 200-event producer execution. Later validation batches also used --count 50 and --count 100.

Verify Event Arrival with receive_events.py

Run from the project root

```
python producer\receive_events.py --max-events 20 --starting-position -1
```

- The receiver uses EVENT_HUB_CONNECTION_STRING, EVENT_HUB_NAME, and EVENT_HUB_CONSUMER_GROUP from .env.
- starting-position -1 reads from the beginning of retained events for verification.
- Stop with Ctrl+C if you run without a max-events limit.

Local consumer output confirms shipment telemetry can be read from Event Hubs.

Producer Test Flags - Create Duplicates and Late Events

Recommended data-quality test batch

```
python producer\send_events.py --count 200 --delay 0.15 \
  --duplicate-every 25 --late-every 40
```

- Every 25th event attempt can reuse the previous event_id so Silver deduplication can be demonstrated.
- Every 40th event can use an event_ts about 20 minutes old so watermark/late-event behavior can be demonstrated.
- region is used as the Event Hubs partition key in the producer code.

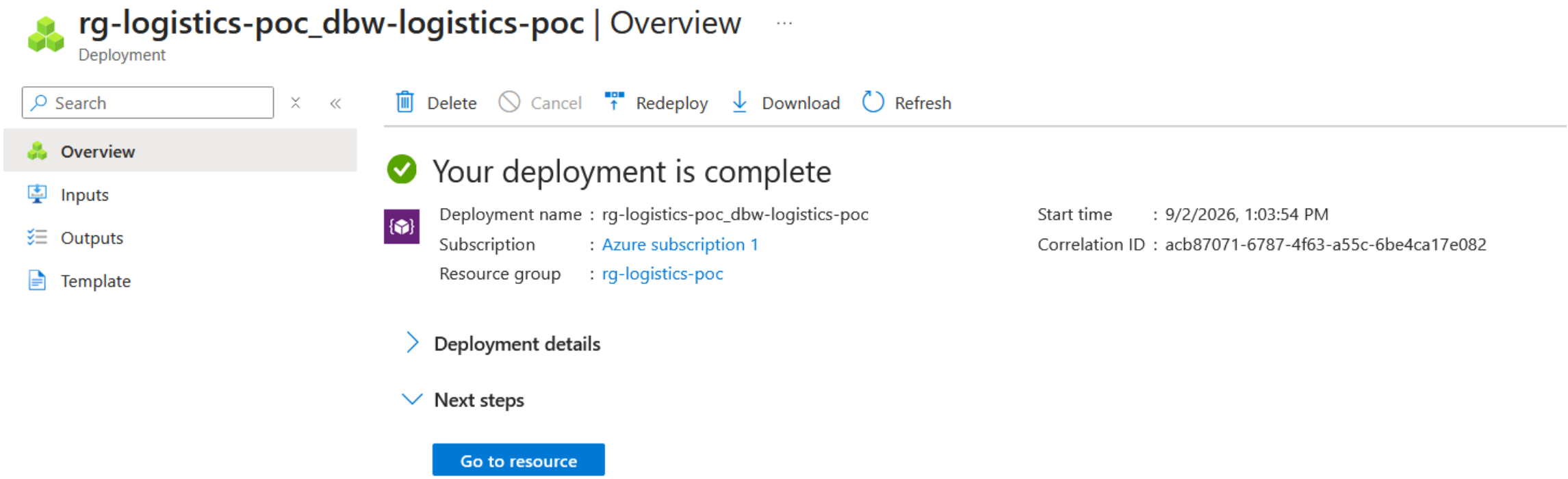
Step 5 - Create Azure Databricks

- Azure portal -> Azure Databricks -> Create.
- Use the existing POC resource group.
- The completed run used a dbw-logistics-poc style workspace name.
- Keep the workspace in the same region as the POC resources where practical.
- Create, then select Launch Workspace.

Cost

Use the smallest practical compute for your account and stop it when finished. The run shown in later screenshots used Serverless compute.

Evidence - Databricks Workspace Created



Azure Databricks workspace deployment completed successfully.

Databricks Compute Choice - Understand This Before Running

Option	When it fits this POC	Important limitation
Serverless	Convenient and was used by the successful evidence run	Does not permit spark.conf.set with fs.azure.account.key and does not support an infinite interactive stream trigger in this run
All-purpose single-user cluster	Useful if you must configure certain Spark filesystem settings directly	Costs run while the cluster is active; use auto-termination
Unity Catalog external location / volume	Best fit for governed Serverless access	Requires Catalog permissions and correct external/managed storage setup

Import the Databricks Scripts in This Order

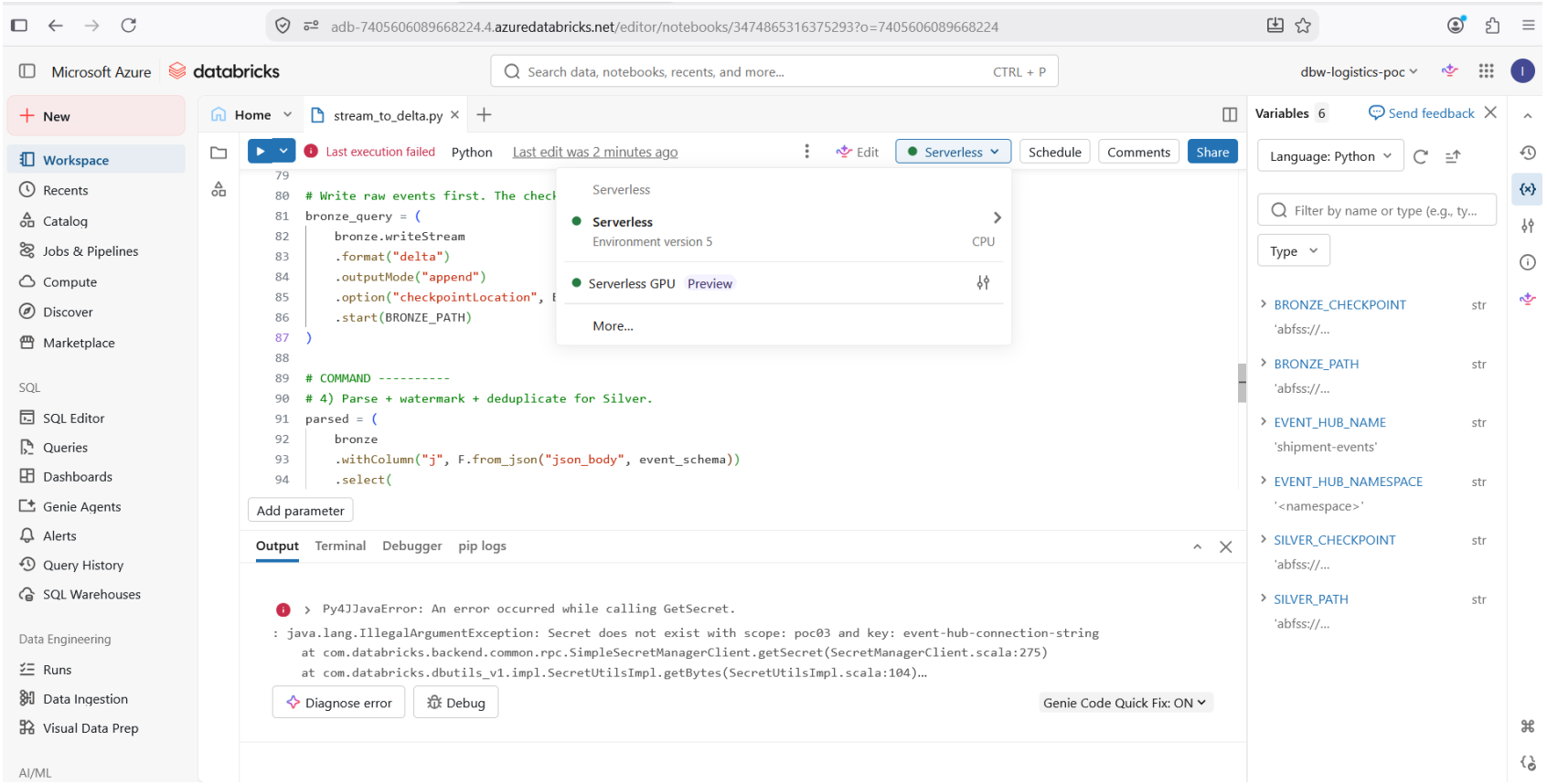
- Workspace -> your user folder -> Import.
- Import databricks/stream_to_delta.py first.
- Import databricks/inspect_and_validate.py second.
- Import databricks/export_gold_parquet.py third.

Logical data flow

Pipeline

```
Event Hubs Kafka endpoint
  -> raw JSON
  -> Bronze Delta
  -> parse schema
  -> event_ts timestamp
  -> watermark + event_id dedup
  -> Silver Delta
  -> Gold KPI Parquet
```

Issue 3 - Databricks Secret Scope Did Not Exist



The imported notebook attempted dbutils.secrets.get(scope="poc03", key="event-hub-connection-string"), but that secret was not configured.

Issue 3 - Databricks Secret Scope Did Not Exist - Continued

Error

Secret does not exist with scope: poc03 and key: event-hub-connection-string.

Fix - Event Hubs Credential Options in Databricks

Preferred

- Create/use a Databricks secret scope named poc03.
- Store the Event Hubs connection string with key event-hub-connection-string.
- Keep dbutils.secrets.get(...) in the notebook.

POC-only emergency path used during troubleshooting

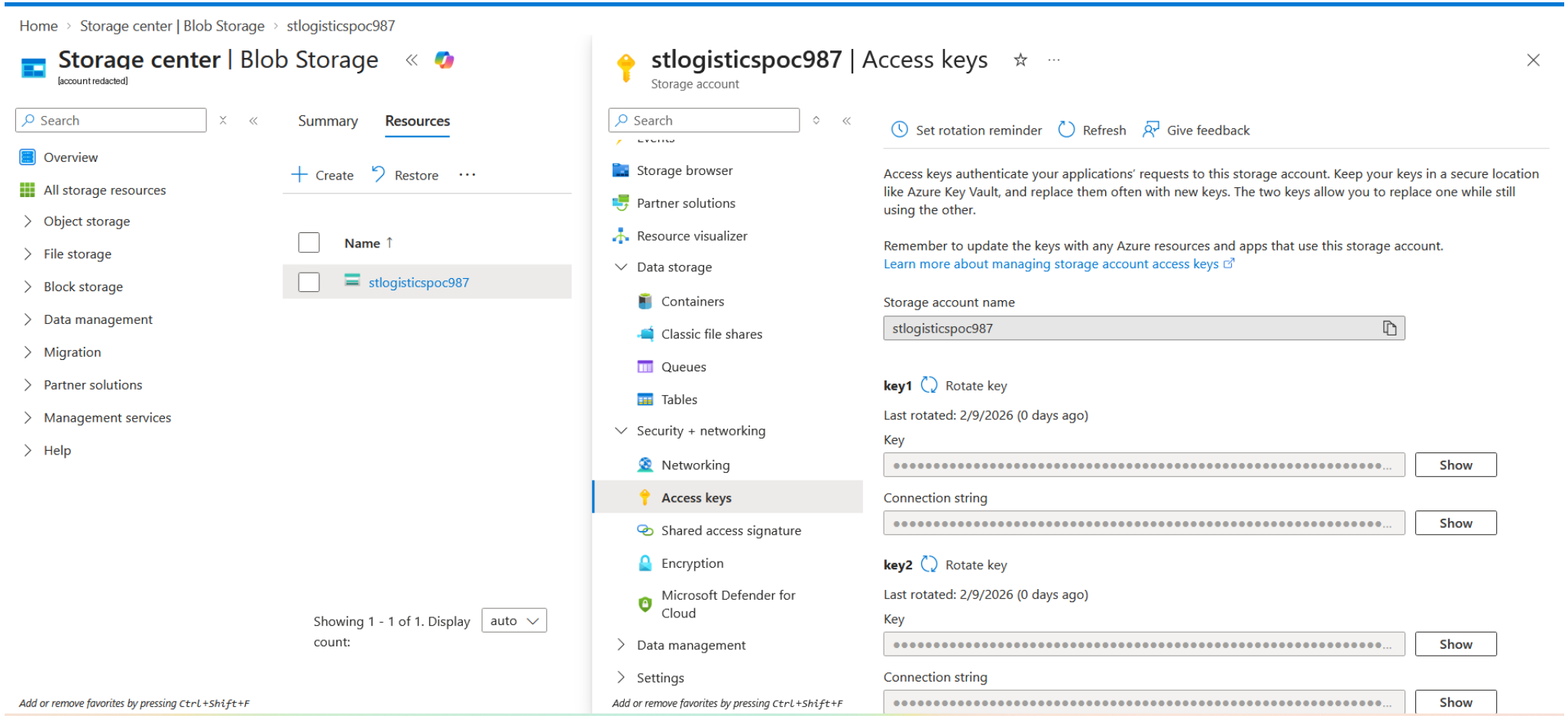
Use placeholders in saved code

```
EVENT_HUB_CONNECTION_STRING = (
    "Endpoint=sb://eh-logistics-poc-dev.servicebus.windows.net/;"
    "SharedAccessKeyName=<policy>;SharedAccessKey=<secret>"
)
```

Security warning

Do not commit a real connection string. If a secret was exposed in code or a screenshot, rotate it.

Storage Access Key - Where It Was Found



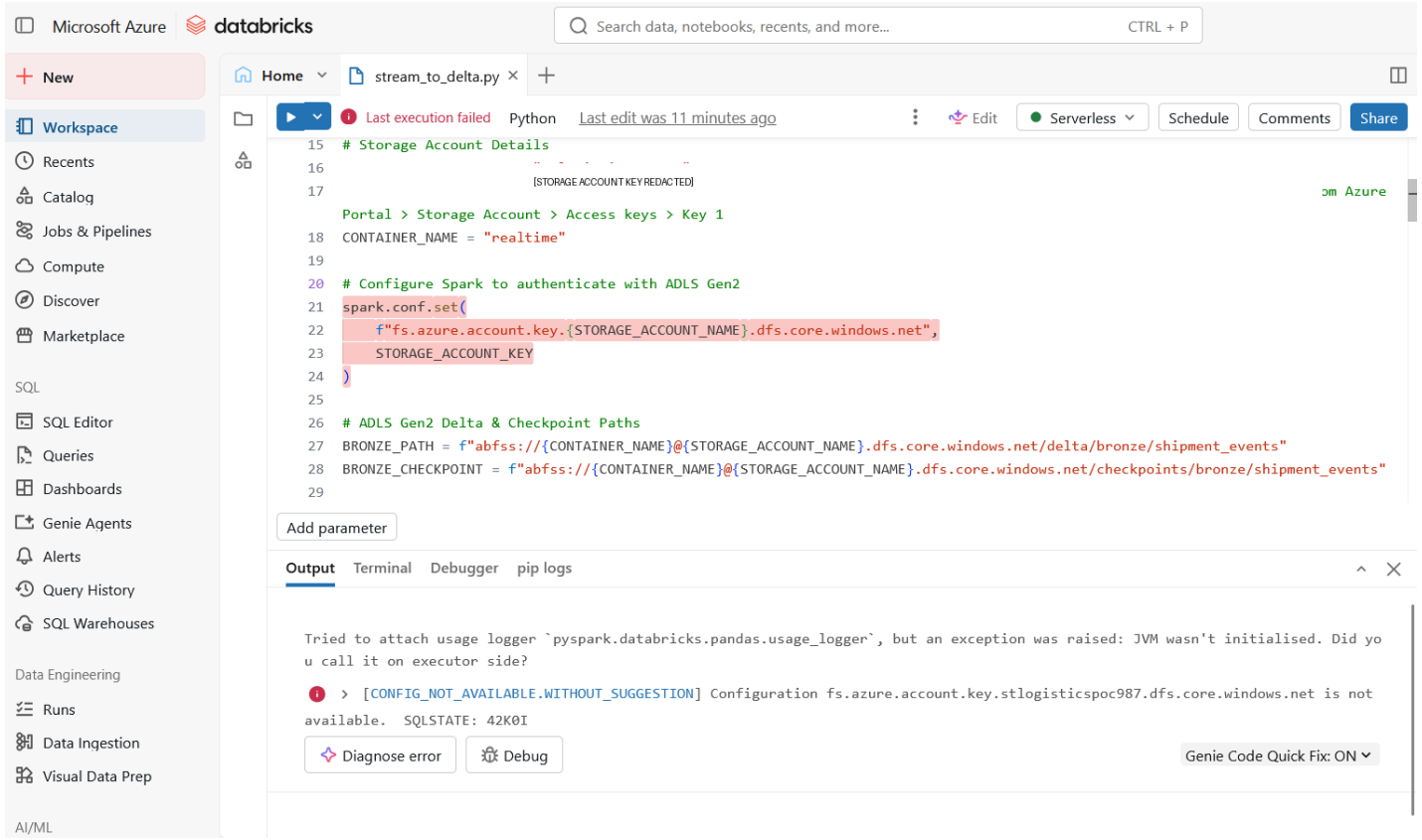
Azure Storage access-key screen used during troubleshooting. Account identity information is redacted.

Storage Access Key - Where It Was Found - Continued

Do not copy this design into production

The storage key path was used only while diagnosing the POC. Serverless later rejected runtime fs.azure.account.key configuration, and the working path moved to Unity Catalog Volume storage.

Issue 4 - JVM Was Not Initialized / CONFIG_NOT_AVAILABLE



The actual storage key shown in the original screenshot has been redacted. The important error is CONFIG_NOT_AVAILABLE for fs.azure.account.key on Serverless.

Issue 4 - JVM Was Not Initialized / CONFIG_NOT_AVAILABLE - Conti

Root cause

Serverless compute did not allow spark.conf.set("fs.azure.account.key..dfs.core.windows.net", ...). The JVM warning was secondary to this configuration failure.

Fix - Choose One Storage Access Pattern

Option A - All-purpose single-user compute

- Attach the notebook to a non-Serverless single-user cluster.
- Use the supported ADLS authentication pattern for that compute.
- Prefer secrets/managed identity over hardcoded keys.

Option B - Serverless with Unity Catalog (working direction)

- Remove spark.conf.set(fs.azure.account.key...).
- Use a Unity Catalog Volume or a governed external location/credential.
- Grant the appropriate catalog/storage permissions.

What the successful evidence used

The run moved to /Volumes/dbw_logistics_poc/default/realtime, which is why data appeared in Databricks Catalog rather than in the standalone stlogisticspoc987 container.

Final Working Storage Paths for the Serverless Run

Working notebook adaptation from the POC run

```
VOLUME_BASE = "/Volumes/dbw_logistics_poc/default/realtime"

BRONZE_PATH = f"{VOLUME_BASE}/delta/bronze/shipment_events"
SILVER_PATH = f"{VOLUME_BASE}/delta/silver/shipment_events"
GOLD_PATH = f"{VOLUME_BASE}/gold/shipment_summary"

BRONZE_CHECKPOINT = (
    f"{VOLUME_BASE}/checkpoints/v3/bronze/shipment_events"
)
SILVER_CHECKPOINT = (
    f"{VOLUME_BASE}/checkpoints/v3/silver/shipment_events"
)
```

Why v3 appears here

The checkpoint location was changed during troubleshooting after an earlier zero-row attempt had already committed stream progress. For a normal restart test, keep the same working checkpoint; do not keep changing it.

Issue 5 - INFINITE_STREAMING_TRIGGER_NOT_SUPPORTED

Root cause

The Serverless interactive environment in this run did not support an infinite ProcessingTime-style stream. It required a bounded micro-batch trigger.

Bronze fix

Bronze - availableNow

```
bronze_query = (
    bronze.writeStream
    .format("delta")
    .outputMode("append")
    .option("checkpointLocation", BRONZE_CHECKPOINT)
    .trigger(availableNow=True)
    .start(BRONZE_PATH)
)
bronze_query.awaitTermination()
```

Issue 5 Fix - Silver availableNow Trigger

Silver - availableNow

```
silver_query = (
    silver.writeStream
    .format("delta")
    .outputMode("append")
    .option("checkpointLocation", SILVER_CHECKPOINT)
    .trigger(availableNow=True)
    .start(SILVER_PATH)
)

print("Bronze query id:", bronze_query.id)
print("Silver query id:", silver_query.id)
silver_query.awaitTermination()
print("Streaming batch completed successfully!")
```

Behavior change

availableNow=True is a one-time sweep of data available to the stream. It is not a continuously listening background stream. Send the events before you launch the sweep, or run another sweep afterward.

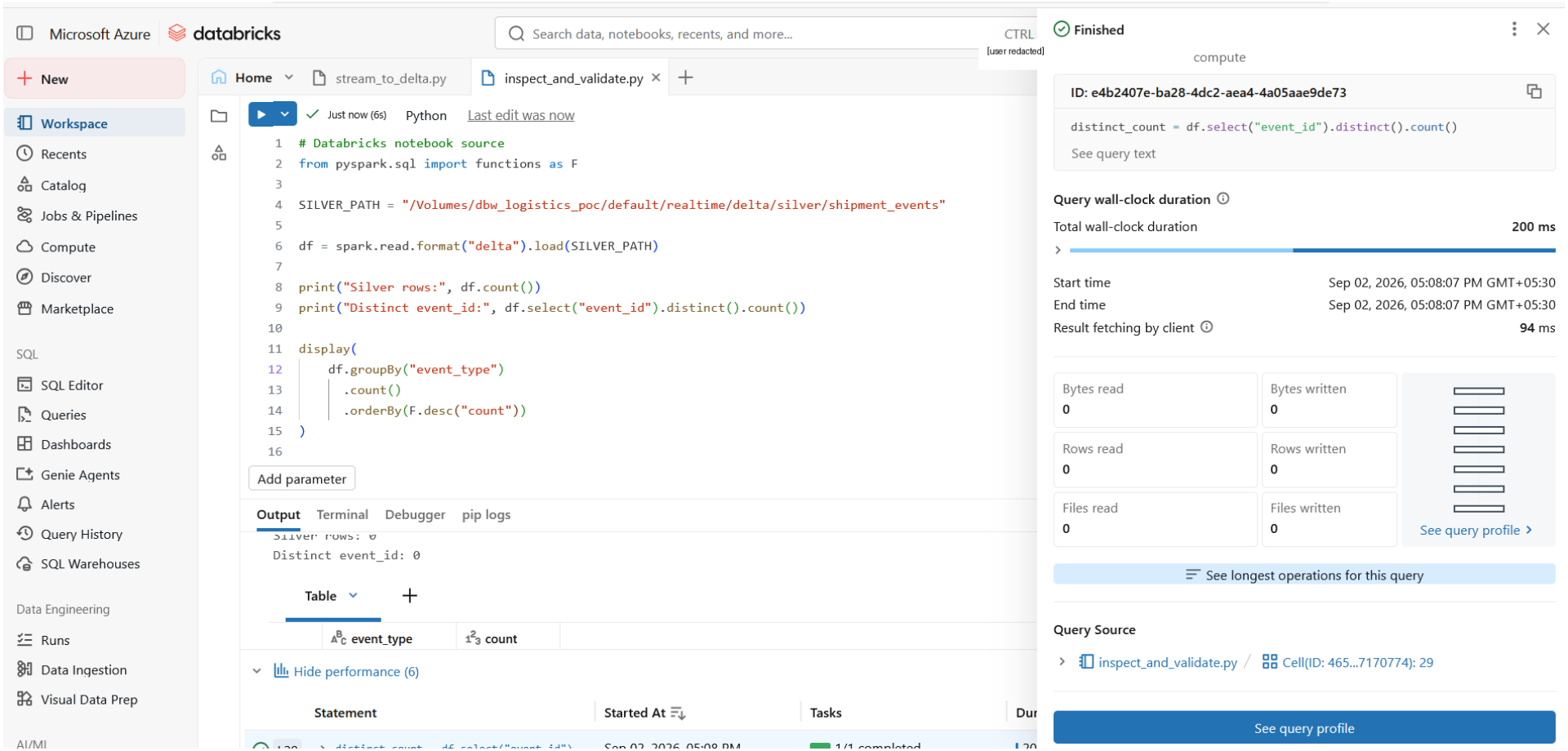
Issue 6 - 0 Rows and an Empty Azure Storage Container

- The notebook had already been changed to /Volumes/... paths, so writes no longer targeted the standalone ADLS container shown in Azure portal.
- availableNow=True checked the source before the new 50-event batch was sent, so that run processed zero available records.
- A previous zero-row execution had already created checkpoint state, so rerunning with the same temporary test state could skip work you expected to see.

Key lesson

Always know both (1) where your output path points and (2) whether your trigger is continuous or bounded.

Evidence - Initial Validation Showed 0 Rows



An intermediate validation attempt showed zero Silver rows. This screenshot is retained because it demonstrates the failure state before the final fix.

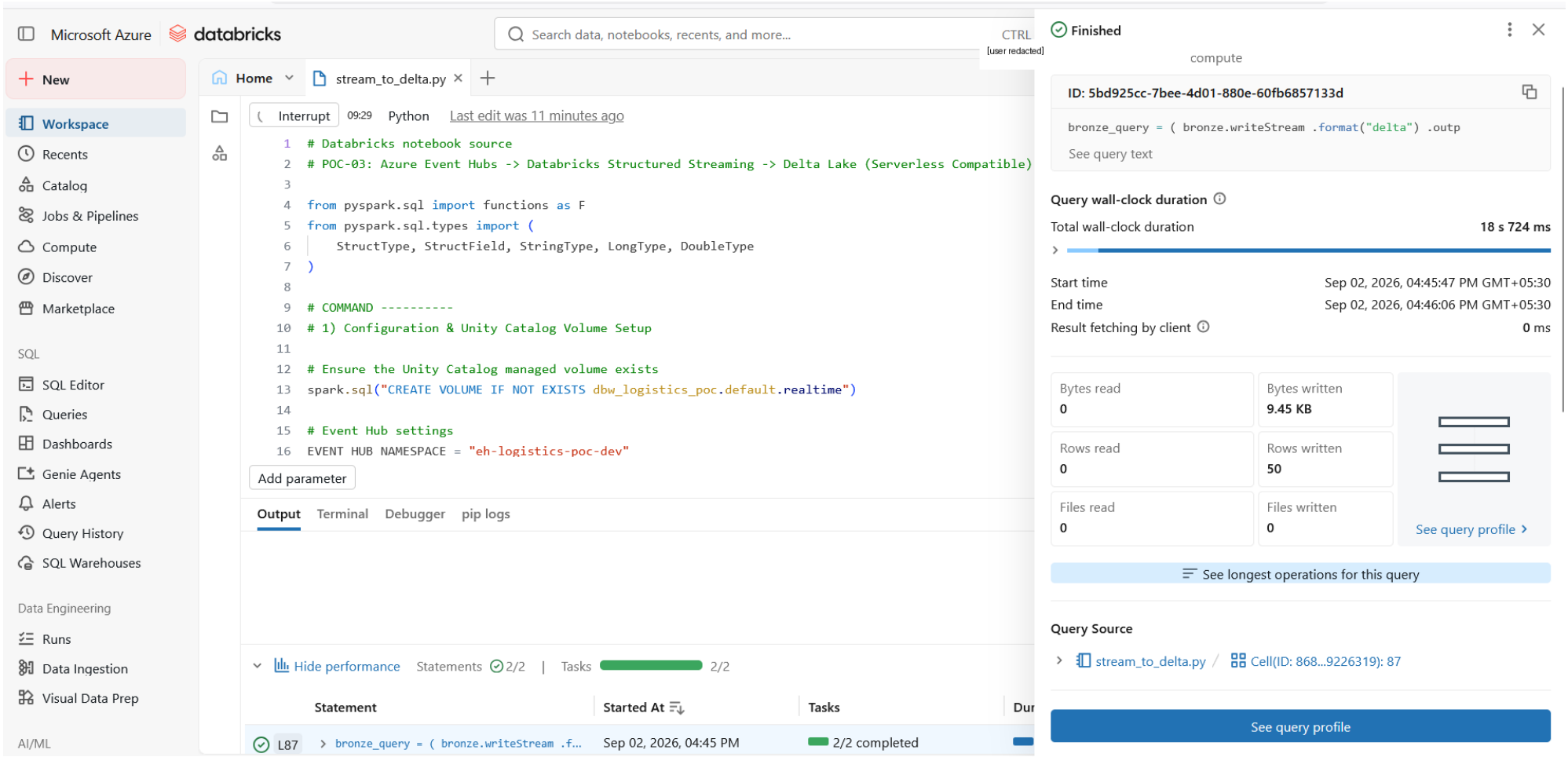
Fix - Correct Order for the Serverless availableNow Test

- Interrupt any notebook cell that is still waiting unexpectedly.
- For the troubleshooting rerun, move to fresh checkpoint paths (v3) only because the earlier test state was disposable.
- Keep Kafka startingOffsets set to earliest for the fresh test path if you want retained messages to be considered.
- Send the test events first.
- Then clear state/outputs if needed and Run all so availableNow processes data that is already waiting.

Troubleshooting sequence

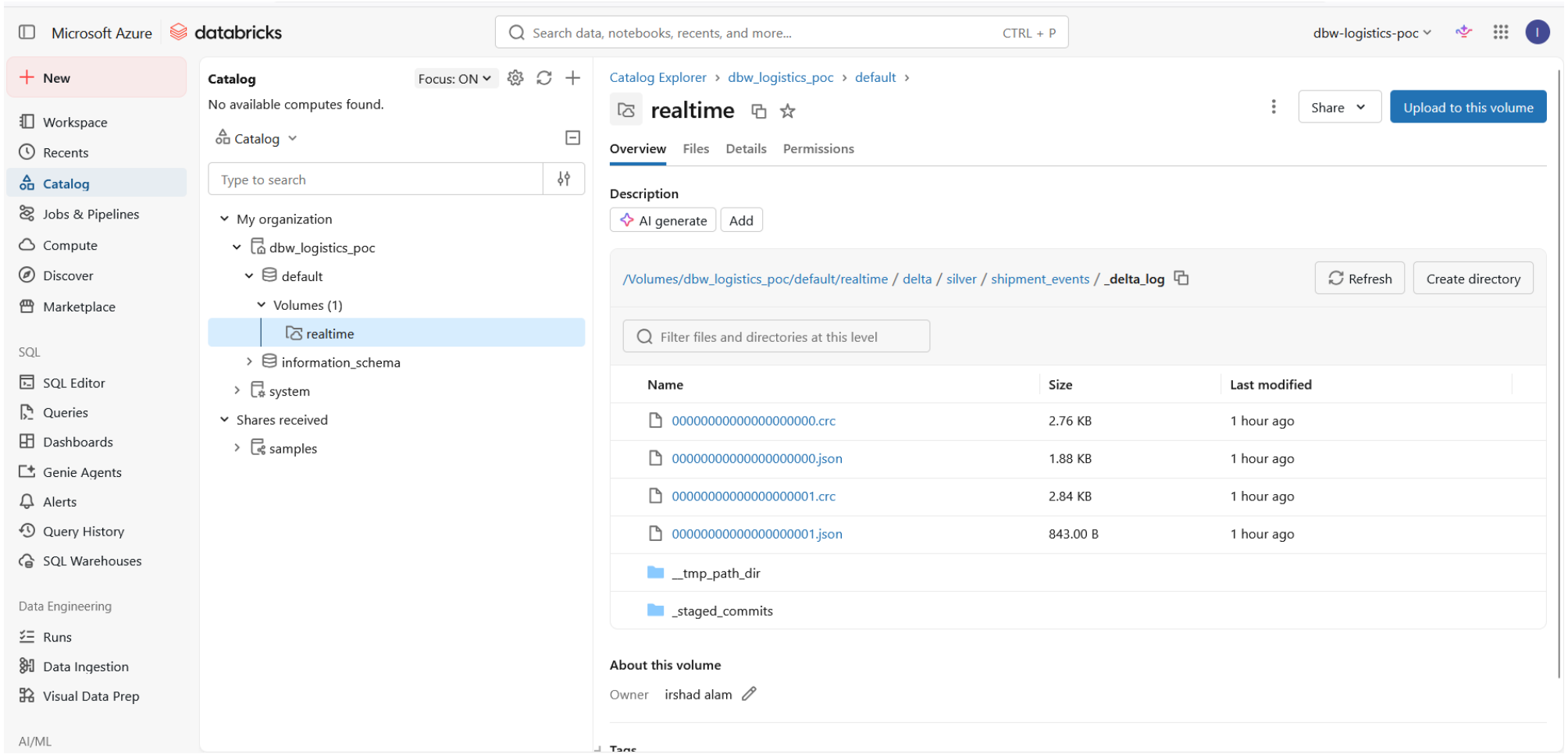
```
python producer\send_events.py --count 50

# Then run stream_to_delta.py with:
"startingOffsets": "earliest"
BRONZE_CHECKPOINT = f"{VOLUME_BASE}/checkpoints/v3/bronze/shipment_events"
SILVER_CHECKPOINT = f"{VOLUME_BASE}/checkpoints/v3/silver/shipment_events"
```

Success - Bronze Stream Wrote 50 Rows



Databricks Serverless query profile shows the Bronze write finishing successfully with 50 rows written. User identity is redacted.

Verify the Files in Databricks Catalog



Catalog Explorer shows the realtime Volume and Delta output folders, including Delta transaction-log and Parquet data files.

Verify the Files in Databricks Catalog - Continued

Why Azure portal looked empty

The successful path shown here is a Databricks Unity Catalog Volume, not the standalone ADLS container.

Bronze vs Silver - What Each Layer Means

Layer	Purpose	POC behavior
Bronze	Raw event capture	Keeps raw JSON plus Event Hubs/Kafka metadata and ingestion timestamp
Silver	Clean event table	Parses schema, converts event_ts, applies watermark/dedup logic
Gold	Business aggregates	Produces regional order/revenue/delay/fulfillment KPIs for SQL/BI

Checkpoint vs watermark

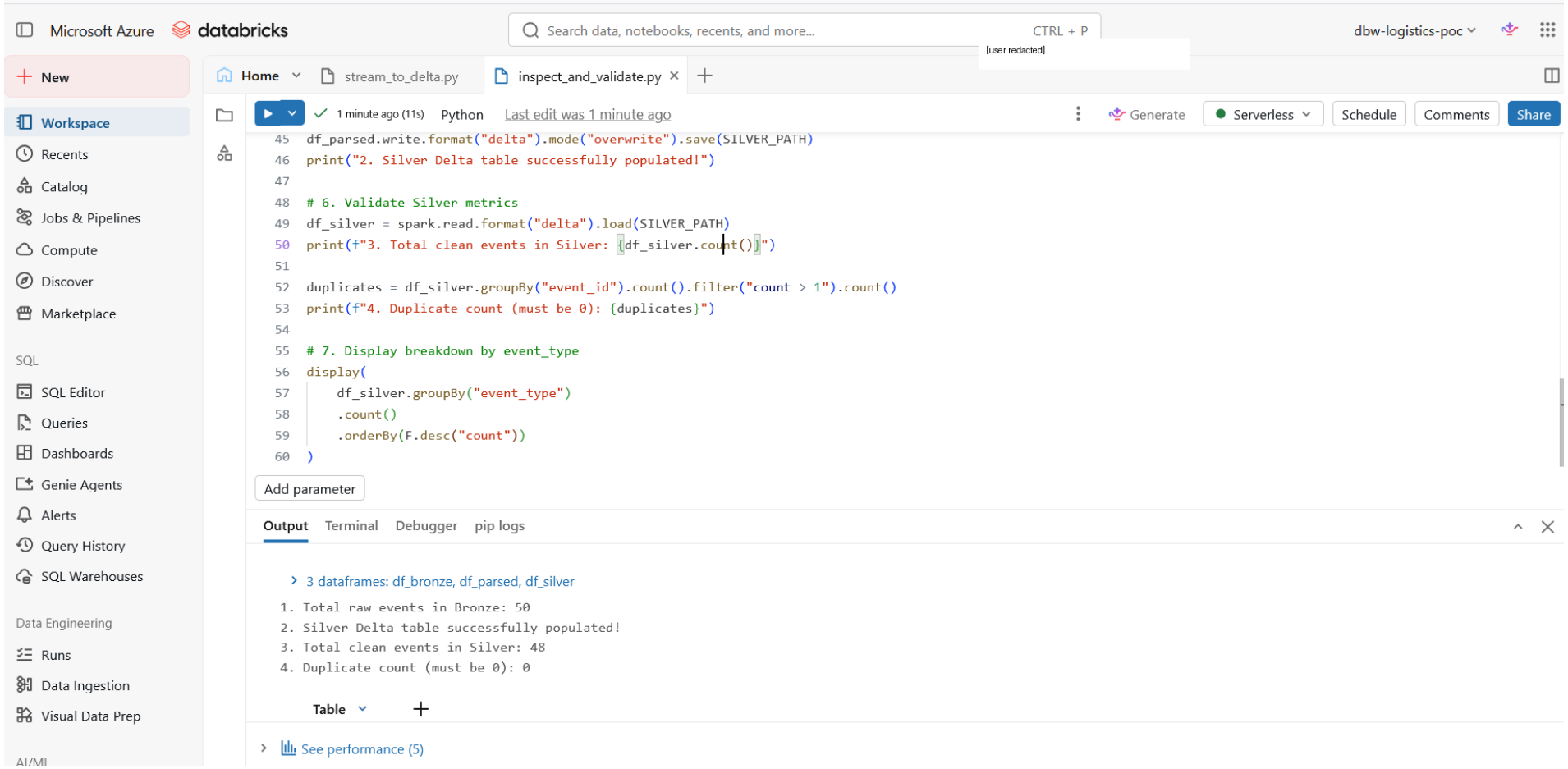
- Checkpoint = recovery/progress/state persistence for the streaming query.
- Watermark = event-time threshold used to bound state for late data.
- They solve different problems and should not be treated as interchangeable.

Run inspect_and_validate.py

Core Silver checks

```
SILVER_PATH = (
    "/Volumes/dbw_logistics_poc/default/realtime/"
    "delta/silver/shipment_events"
)

df = spark.read.format("delta").load(SILVER_PATH)
print("Silver rows:", df.count())
print("Distinct event_id:", df.select("event_id").distinct().count())

duplicates = (
    df.groupBy("event_id").count().filter("count > 1").count()
)
print("Duplicate count (must be 0):", duplicates)
```

Success - Silver Validation Metrics



Final validation evidence: 50 raw Bronze events, Silver populated with 48 clean events, duplicate count 0.

Success - Silver Validation Metrics - Continued

Why 50 became 48

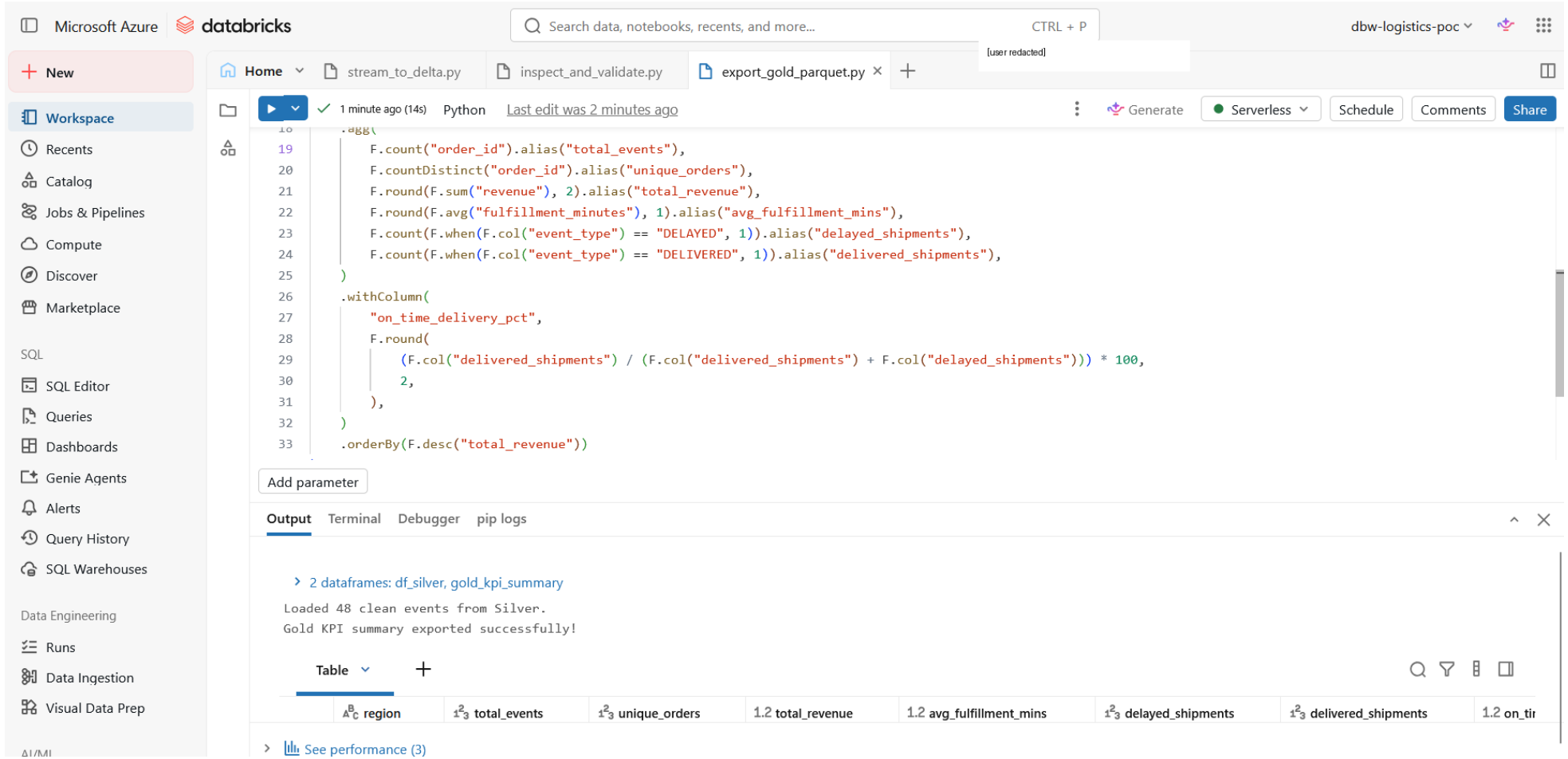
The producer deliberately injected duplicate event_id values. Silver is the curated layer, so the final clean count can be lower than the raw Bronze count.

Build the Gold KPI Layer

Gold aggregation - working Volume path

```
VOLUME_BASE = "/Volumes/dbw_logistics_poc/default/realtime"
SILVER_PATH = f"{VOLUME_BASE}/delta/silver/shipment_events"
GOLD_PATH = f"{VOLUME_BASE}/gold/shipment_summary"

df_silver = spark.read.format("delta").load(SILVER_PATH)

gold_kpi_summary = (
    df_silver.groupBy("region")
    .agg(
        F.count("order_id").alias("total_events"),
        F.countDistinct("order_id").alias("unique_orders"),
        F.round(F.sum("revenue"), 2).alias("total_revenue"),
        F.round(F.avg("fulfillment_minutes"), 1).alias("avg_fulfillment_mins"),
        F.count(F.when(F.col("event_type") == "DELAYED", 1)).alias("delayed_shipments"),
        F.count(F.when(F.col("event_type") == "DELIVERED", 1)).alias("delivered_shipments"),
    )
)

gold_kpi_summary.write.mode("overwrite").parquet(GOLD_PATH)
```

Success - Gold KPI Export



The Gold notebook loaded 48 clean Silver events and exported the KPI summary successfully.

Create SQL Views over Silver and Gold

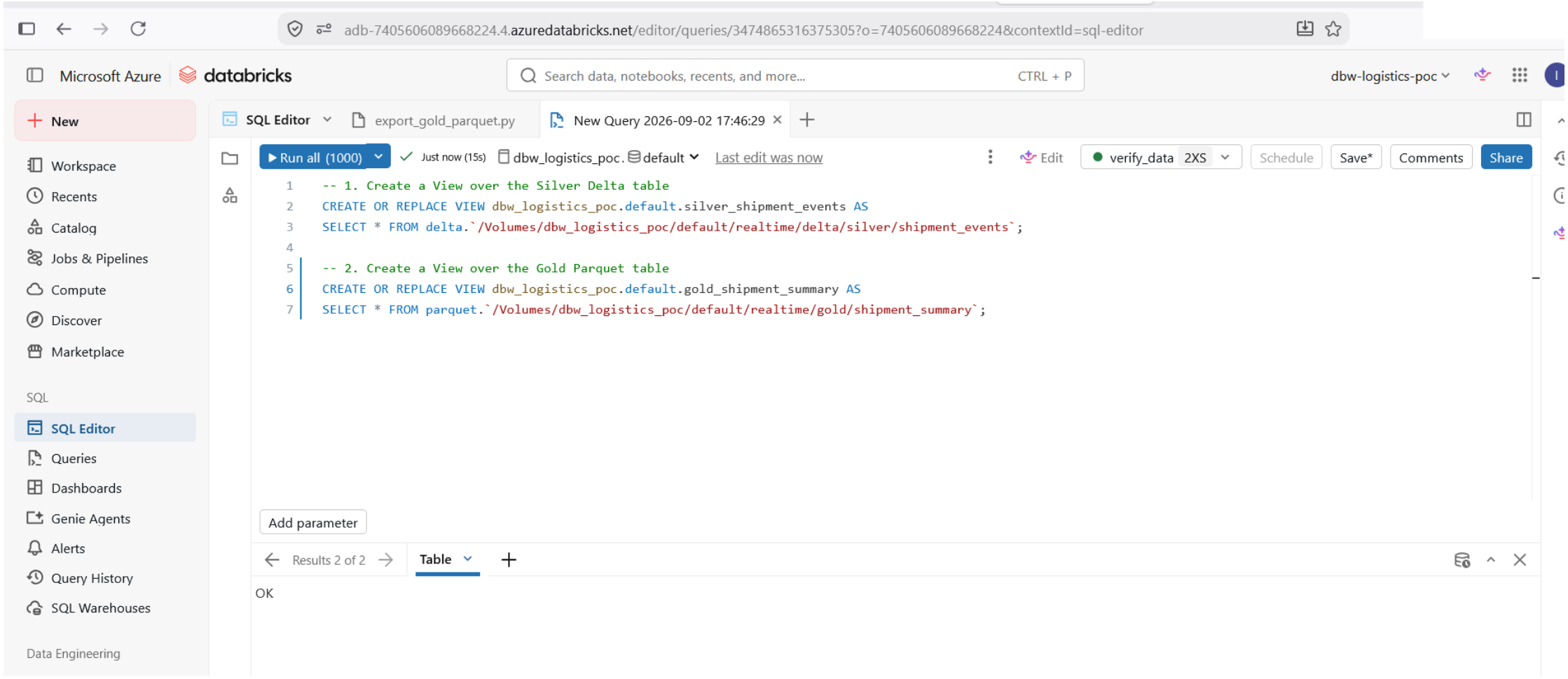
Databricks SQL

```
CREATE OR REPLACE VIEW dbw_logistics_poc.default.silver_shipment_events AS
SELECT * FROM delta.`/Volumes/dbw_logistics_poc/default/realtime/
delta/silver/shipment_events`;

CREATE OR REPLACE VIEW dbw_logistics_poc.default.gold_shipment_summary AS
SELECT * FROM parquet.`/Volumes/dbw_logistics_poc/default/realtime/
gold/shipment_summary`;
```

Why views were used

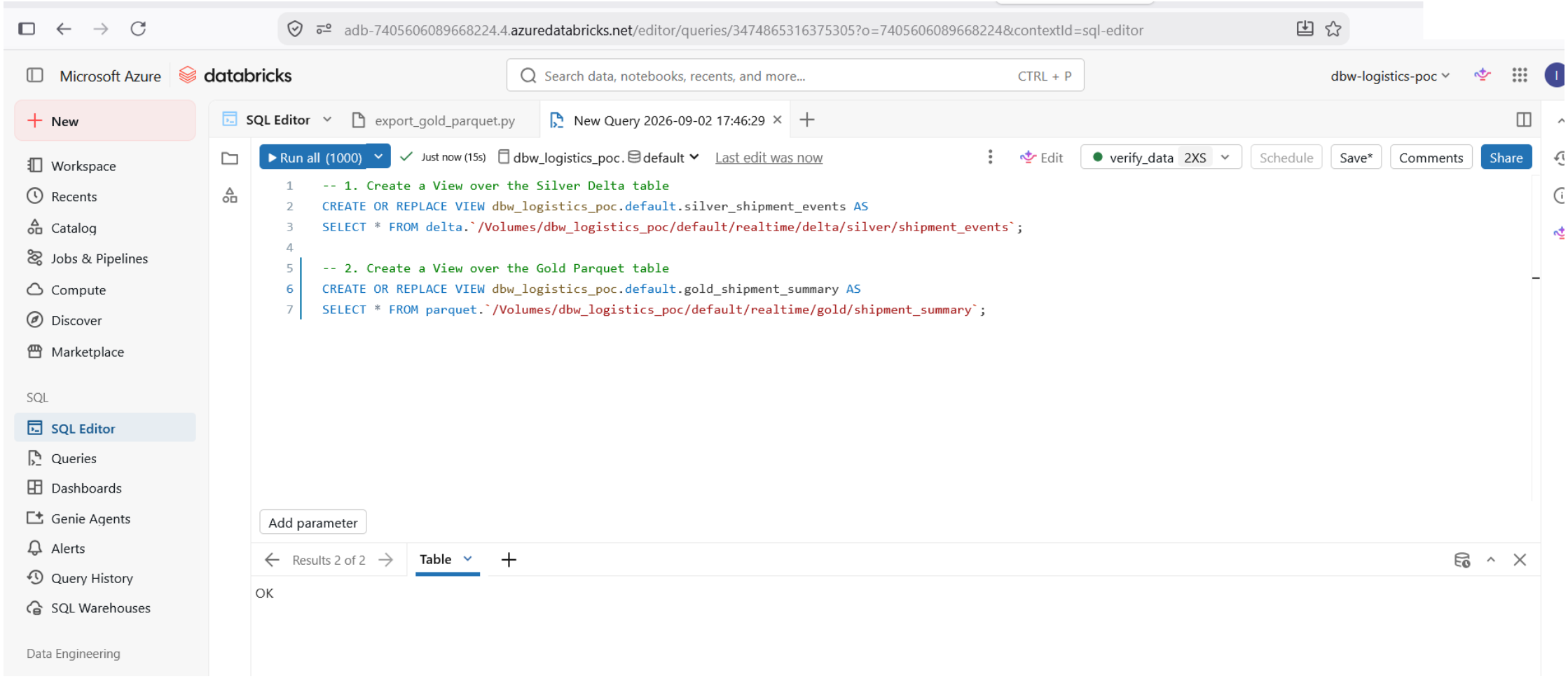
The supplied SQL evidence creates views directly over the Volume file locations so the curated data can be queried from Databricks SQL.

Evidence - SQL Views Created



Databricks SQL Editor shows the Silver Delta and Gold Parquet views being created successfully.

Evidence - Curated Data Registration



Second curated-data evidence screenshot retained from the supplied POC screenshot set.

Query the Gold KPI Summary

Databricks SQL

```
SELECT
    region,
    unique_orders,
    total_revenue,
    avg_fulfillment_mins,
    delayed_shipments,
    delivered_shipments,
    on_time_delivery_pct
FROM dbw_logistics_poc.default.gold_shipment_summary
ORDER BY total_revenue DESC;
```

Use this query to validate the final regional KPI dataset after Gold export. If your Gold implementation does not contain on_time_delivery_pct, add that computed column in the Gold notebook before running this exact projection.

Query Detailed Silver Telemetry

Databricks SQL

```
SELECT
    event_type,
    region,
    count(*) AS event_count,
    round(avg(revenue), 2) AS avg_revenue,
    min(event_ts) AS earliest_event,
    max(event_ts) AS latest_event
FROM dbw_logistics_poc.default.silver_shipment_events
GROUP BY event_type, region
ORDER BY event_count DESC;
```

Pass condition

The query returns grouped counts/timestamps from the deduplicated Silver layer without duplicate event_id values.

Restart and Checkpoint Test

- Record the current Silver row count.
- Stop the streaming notebook/query.
- Send a new small batch, for example 20 events.
- Restart using the SAME working checkpoint paths.
- Wait for the new batch to complete.
- Run Silver validation again.

New batch

```
python producer\send_events.py --count 20 --delay 0.15
```

Critical distinction

For the intentional recovery test, keep the checkpoint. Changing/deleting the checkpoint creates a different stream history and is not a valid demonstration of normal restart recovery.

Fabric Branch - Workspace, Eventhouse and Eventstream

Evidence status

These steps are present in the repository README/setup notes. The supplied screenshot set does not contain Fabric evidence, so this section is a repeatable reference branch rather than screenshot-validated proof.

- Create a Fabric-enabled workspace or Trial workspace if your tenant allows it.
- Create an Eventhouse and use its KQL database.
- Create the ShipmentEvents KQL table.
- Create an Eventstream and add Azure Event Hubs as the source.
- Select the shipment-events Event Hub and appropriate connection/consumer group.
- Add the Eventhouse/KQL table as the destination.
- Map the JSON fields, then Publish the Eventstream.
- Verify live data and run ShipmentEvents | take 10.

Fabric KQL Table and JSON Mapping

KQL - representative mapping from repository

```
.create table ShipmentEvents (
    event_id:string,
    order_id:long,
    event_type:string,
    event_ts:datetime,
    region:string,
    revenue:real,
    fulfillment_minutes:long,
    producer_seq:long
)

.create-or-alter table ShipmentEvents ingestion json mapping
'ShipmentEventsJsonMapping'
'[
  {"column":"event_id","path":"$.event_id","datatype":"string"},
  {"column":"order_id","path":"$.order_id","datatype":"long"},
  {"column":"event_type","path":"$.event_type","datatype":"string"},
  {"column":"event_ts","path":"$.event_ts","datatype":"datetime"},
  {"column":"region","path":"$.region","datatype":"string"}
]'
```

Fabric KQL Verification Queries

KQL

```
ShipmentEvents
| take 20

ShipmentEvents
| summarize events=count() by bin(event_ts, 5m), event_type
| order by event_ts desc

ShipmentEvents
| where event_type == "DELAYED"
| summarize delayed=count() by region
| order by delayed desc
```

Freshness and duplicate detection

KQL

```
ShipmentEvents
| summarize latest_event=max(event_ts)
| extend freshness_seconds = datetime_diff("second", now(), latest_event)
```

Fabric KQL Verification Queries - Continued

```
ShipmentEvents
| summarize copies=count() by event_id
| where copies > 1
| order by copies desc
```

Fabric OneLake / Lakehouse Path

- Create a Lakehouse in the same Fabric workspace.
- Where your tenant/authentication supports it, create a shortcut to the curated ADLS Gold path rather than copying the entire dataset.
- If a shortcut is not practical, load only the tiny Gold dataset for the learning POC and document that limitation.
- Use a Fabric Notebook, Dataflow Gen2, or Pipeline to create a small curated table.
- Compare the curated Fabric totals against the Databricks Gold result.

Shortcut concept

A OneLake shortcut exposes supported external data without creating an unnecessary second physical copy.

Synapse Serverless SQL - Important Path Requirement

Architecture fork

The repository Synapse SQL is written for Gold Parquet stored in ADLS. The screenshot-validated successful run stored Gold in a Databricks managed Volume. To execute Synapse exactly as designed, also export/copy Gold to the ADLS realtime/gold/shipment_summary path and grant Synapse read access.

- Create an Azure Synapse workspace.
- Use the built-in serverless SQL pool only.
- Do not create a dedicated SQL pool for this POC.
- Ensure the signed-in identity has read permission on the target ADLS path.

Synapse SQL - Smoke Test over Gold Parquet

Synapse serverless SQL

```
SELECT TOP 100 *
FROM OPENROWSET(
    BULK 'https://<storage-account>.dfs.core.windows.net/
          realtime/gold/shipment_summary/*.parquet',
    FORMAT = 'PARQUET'
) AS rows;
```

Pass condition

Rows are returned from the ADLS Gold Parquet folder using the built-in serverless SQL pool.

Synapse SQL - Cost-Aware Projection

Synapse serverless SQL

```
SELECT
    region,
    total_orders,
    delayed_shipments,
    revenue
FROM OPENROWSET(
    BULK 'https://<storage-account>.dfs.core.windows.net/
          realtime/gold/shipment_summary/*.parquet',
    FORMAT = 'PARQUET'
)
WITH (
    region VARCHAR(50),
    total_orders BIGINT,
    revenue FLOAT,
    delayed_shipments BIGINT
) AS gold
ORDER BY region;
```

Synapse SQL - Cost-Aware Projection - Continued

Cost lesson

Inspect data processed in query details and compare a broad SELECT * with the narrower projection. The POC dataset is tiny, so the measured difference may also be tiny.

Synapse SQL - Business Totals

Synapse serverless SQL

```
SELECT
    SUM(total_orders) AS total_orders,
    ROUND(SUM(revenue), 2) AS revenue,
    SUM(delayed_shipments) AS delayed_shipments
FROM OPENROWSET(
    BULK 'https://<storage-account>.dfs.core.windows.net/
          realtime/gold/shipment_summary/*.parquet',
    FORMAT = 'PARQUET'
)
WITH (
    total_orders BIGINT,
    revenue FLOAT,
    delayed_shipments BIGINT
) AS gold;
```

Semantic Model Measures

DAX

```
Total Orders =
SUM('shipment_summary_fabric'[total_orders])

Revenue =
SUM('shipment_summary_fabric'[revenue])

Delayed Shipments =
SUM('shipment_summary_fabric'[delayed_shipments])

Average Fulfillment Time =
AVERAGE('shipment_summary_fabric'[avg_fulfillment_minutes])
```

- Create four KPI cards/measures.
- Compare the totals with your authoritative curated Gold layer.
- If Eventhouse raw data contains duplicates, do not expect raw real-time totals to match deduplicated Gold automatically.

Monitoring Checklist

Area	What to capture
Event Hubs	Incoming Messages, Incoming Bytes, throttling/errors if any
Databricks stream	Rows processed/written, batch duration, query status, checkpoint path
Fabric	Eventstream source/destination health and latest KQL event timestamp
Freshness	Maximum event_ts compared with current time
Transforms	At least one successful run and one captured/recovered failure
SQL	Gold query result plus data-scanned/processed evidence where available

Failure and Recovery Evidence - What to Save

- Local producer failure showing the malformed/blank connection-string issue.
- Successful producer output.
- Successful receiver output.
- Databricks missing-secret error.
- Serverless CONFIG_NOT_AVAILABLE/JVM error with secret material redacted.
- Intermediate zero-row validation state.
- Successful 50-row Bronze write.
- Final 48-row Silver / 0-duplicate validation.
- Successful Gold export.
- SQL view/query evidence.

Security

Never save or publish screenshots containing storage keys, SAS tokens, Event Hubs connection strings, or personal account identifiers. Rotate any credential that was exposed during troubleshooting.

Purview / Governance Reference

- Purview is optional in this POC because account availability, permissions, and cost vary.
- If available, register only the relevant POC sources and scan the smallest practical scope.
- Review discovered assets, schema, classifications, and lineage where connectors expose it.
- If Purview is not deployed, document the intended enterprise design and clearly mark it design-only.
- Catalog/governance does not replace runtime access control: Azure RBAC/ACLs, Fabric permissions, Databricks Unity Catalog, and SQL permissions still matter.

Common Troubleshooting Reference

Symptom	Check / fix
azure.eventhub module missing	Activate .venv and pip install -r requirements.txt
Receiver prints nothing	Send data, verify Event Hub name/connection, try starting-position -1
Kafka auth error	Namespace, :9093, SASL PLAIN, SASL_SSL, Listen rights, full connection string
dropDuplicatesWithinWatermark unavailable	Use withWatermark(...).dropDuplicates(["event_id"]) and document runtime difference
Cannot write abfss://	Check storage auth/RBAC/external location/credential and exact path
Synapse OPENROWSET denied	Check ADLS read permission, identity, dfs.core.windows.net path and file existence
Fabric Eventstream empty	Verify Event Hubs messages, connection, source, destination mapping and Publish state

Cleanup - Stop Cost When the POC Is Finished

Before deleting

- Save screenshots and query results.
- Record validation counts and Gold output.
- Stop Databricks compute / SQL warehouse.
- Save Fabric evidence if that branch was executed.

Delete Azure POC resources

Azure CLI option from the raw run notes

```
az group delete --name rg-logistics-poc --yes --no-wait
```

- Or delete the dedicated resource group from Azure portal.
- Delete Fabric POC items/workspace separately if they are not tied to the Azure resource group.
- Delete Purview/Log Analytics only if those resources were created solely for this POC and are not shared.

Exact End-to-End Command Sequence - First Repeat Run

Runbook

```
# 1. Local environment
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env

# 2. After .env is populated
python producer\send_events.py --count 20 --delay 0.2
python producer\receive_events.py --max-events 10 --starting-position -1

# 3. Data-quality batch
python producer\send_events.py --count 200 --delay 0.15 \
  --duplicate-every 25 --late-every 40

# 4. For Serverless availableNow, send before launching the sweep
python producer\send_events.py --count 50

# 5. In Databricks run in order
stream_to_delta.py
inspect_and_validate.py
```

Exact End-to-End Command Sequence - First Repeat Run - Continue

```
export_gold_parquet.py

# 6. Run Databricks SQL view/query statements
# 7. Execute Fabric/Synapse branches if required
```

Final Validation Checklist

Component	Done when...
Resource group	POC resources are grouped together
Event Hubs	Producer sends and receiver reads
Bronze	Raw stream write succeeds and files/Delta log exist
Silver	Rows > 0 and duplicate event_id count is 0
Checkpoint	Restart uses the same working checkpoint and resumes correctly
Gold	Regional KPI Parquet is exported
Databricks SQL	Silver/Gold view queries run successfully
Fabric	If executed: Eventstream -> Eventhouse and KQL queries work
Synapse	If executed: serverless OPENROWSET reads ADLS Gold
Monitoring	Arrival, stream health, freshness, and one failure/recovery are captured
Cleanup	Compute stopped and POC-only resources removed when no longer needed

Interview Q&A - Streaming Concepts

Q1. What is the Event Hubs partition-key trade-off?

Using a stable partition key can preserve ordering for related events, but a highly skewed key can create a hot partition and uneven throughput.

Q2. What is a watermark?

A watermark is an event-time threshold used by Structured Streaming to bound how long state is kept while waiting for late data.

Q3. Checkpoint vs watermark?

Checkpoint persists stream progress/recovery state. Watermark controls event-time lateness/state retention. They solve different problems.

Q4. Why did Bronze have 50 rows while Silver had 48?

The producer intentionally created duplicate event_id values. Bronze retained the raw attempts; Silver applied deduplication and produced the clean curated count.

Interview Q&A - Serving, Fabric and Governance

Q5. Eventhouse/KQL vs Lakehouse?

Eventhouse/KQL is optimized for fast event/time-series ingestion and low-latency queries. A Lakehouse is oriented toward open lake storage, Spark/SQL analytics, and broader batch/ML/BI workflows.

Q6. When would you use Synapse serverless SQL?

For on-demand SQL over lake files when you do not want to provision a dedicated SQL pool for the workload.

Q7. OneLake shortcut vs copy?

A shortcut references accessible data without an unnecessary physical duplicate. A copy creates another dataset with its own storage and refresh lifecycle.

Q8. How do you monitor freshness?

Track the latest business event timestamp and compare it with current time, then combine that signal with source-ingestion and stream-processing health.

Beginner Completion Summary

- Start simple: prove Event Hubs locally before adding Databricks.
- Keep credentials out of code and screenshots; use secrets/managed identities for durable implementations.
- On Databricks Serverless, use governed storage patterns rather than runtime fs.azure.account.key configuration.
- Understand availableNow=True: it processes what is available and then finishes.
- Keep checkpoints for true restart tests; use a fresh checkpoint only for an intentional clean troubleshooting experiment.
- Treat Bronze as raw evidence, Silver as clean data, and Gold as business-ready aggregates.
- If you want Synapse serverless to query the result, ensure the Gold Parquet exists in ADLS rather than only in a Databricks managed Volume.
- Stop compute and clean up resources when the learning exercise is complete.

Completed evidence result

The supplied POC evidence reached a successful 50-row Bronze write, 48 clean Silver events, duplicate count 0, Gold KPI export, and Databricks SQL view creation.